UNIVERSIDADE FEDERAL DO PARÁ

INSTITUTO DE CIÊNCIAS EXATAS E NATURAIS

FACULDADE DE COMPUTACAO

TAREFA AVALIATIVA

ESTRUTURAS DE DADOS

Pilhas, Árvores Binárias de Busca e Árvores AVL

Valor da atividade: 5,0 pontos

Aluno(a): _____

Aluno(a): _____

Aluno(a): _____

Turma: _____ Data: ____/____/_____

ORIENTAÇÕES GERAIS DA TAREFA

Estruturas de Dados – Pilhas, Árvores Binárias de Busca e Árvores AVL

Esta atividade tem como objetivo aplicar, de forma prática, os conceitos de estruturas de dados, alocação dinâmica, recursão, árvores e análise de algoritmos. A tarefa deverá ser desenvolvida com atenção especial à implementação correta das operações e à demonstração do funcionamento das estruturas.

1. Valor da atividade

A atividade vale, no máximo, 5,0 (cinco) pontos. A nota será atribuída considerando a correção das soluções, a implementação dos algoritmos, a qualidade da justificativa e o cumprimento das orientações de entrega.

2. Formação das equipes

A atividade poderá ser realizada individualmente ou em equipe de, no máximo, 3 (três) alunos. Todos os integrantes da equipe devem participar efetivamente do desenvolvimento e ser capazes de explicar as soluções apresentadas.

3. Forma obrigatória de entrega

• A solução final deverá ser entregue obrigatoriamente em arquivo PDF.

• O PDF deverá conter as respostas de todas as questões, as explicações, os cálculos, as árvores/desenhos solicitados e os códigos-fonte desenvolvidos.

• Os códigos-fonte devem estar incluídos no próprio PDF, preferencialmente em fonte monoespaçada e com boa legibilidade.

• Quando a questão solicitar implementação em C, não basta apresentar apenas o resultado: o código-fonte correspondente deverá fazer parte da entrega.

• ATENÇÃO: soluções entregues sem os respectivos códigos-fonte não serão pontuadas nos itens que exigem implementação.

• O arquivo deve permitir identificar claramente os integrantes da equipe.

4. Orientações para resolver as questões

• Leia todo o enunciado antes de iniciar a implementação.

• Desenvolva cada operação solicitada de acordo com a estrutura de dados indicada no enunciado.

• Para questões que envolvem árvores, apresente as transformações passo a passo, sempre que solicitado.

• Nas questões de implementação, teste as funções com os dados fornecidos no enunciado e registre os resultados.

• Não apresente somente a resposta final. Demonstre o raciocínio utilizado para chegar ao resultado.

• Utilize comentários no código para identificar as principais partes da implementação.

• Verifique se o código compila e executa corretamente antes de gerar o PDF.

• Nas árvores AVL, o balanceamento deve ocorrer dinamicamente por meio das rotações apropriadas; não é permitido simplesmente reconstruir a árvore.

5. Competências e habilidades exploradas

• Implementar estruturas de dados utilizando linguagem C.

• Utilizar ponteiros, estruturas (struct), alocação dinâmica e gerenciamento de memória.

• Aplicar recursão na implementação e análise de operações sobre estruturas hierárquicas.

• Compreender e aplicar os princípios LIFO em pilhas.

• Construir, percorrer, buscar, inserir e remover elementos em Árvores Binárias de Busca.

• Compreender e aplicar balanceamento de Árvores AVL e as rotações LL, RR, LR e RL.

• Analisar altura, fatores de balanceamento e complexidade assintótica dos algoritmos.

• Desenvolver capacidade de depuração, teste e validação de programas.

• Comunicar soluções algorítmicas de forma clara, organizada e tecnicamente fundamentada.

6. Distribuição da pontuação

Questão	Tema	Valor máximo
1	Pilha sequencial – implementação e operações PUSH/POP	1,00 ponto
2	Pilha – análise das permutações possíveis	0,75 ponto
3	Árvore Binária de Busca – implementação, remoção e análise	1,50 ponto
4	Árvore AVL – inserção, rotações, remoção e análise	1,75 ponto
TOTAL	Atividade completa	5,00 pontos

7. Critérios gerais de correção

Critério	O que será observado
Correção conceitual	A solução demonstra domínio dos conceitos da estrutura de dados solicitada.
Implementação	As funções solicitadas estão implementadas corretamente e o código é coerente com o enunciado.
Execução e resultados	Os resultados apresentados correspondem às operações realizadas.
Justificativas	As decisões algorítmicas e as transformações das estruturas são explicadas.

Organização	O PDF está organizado, legível e identifica claramente cada questão e seus itens.

IMPORTANTE: a pontuação máxima da atividade é 5,0 pontos. O não atendimento às exigências de implementação e entrega poderá comprometer a pontuação dos respectivos itens. Em especial, questões que exigem código-fonte não serão pontuadas quando o código não estiver incluído no PDF.

1) Projete uma estrutura do tipo pilha (stack/LIFO) sequencial de 10 espaços para guardar informações de páginas da web visitadas. Essas informações estarão contidas na seguinte struct:

```
#define MAX 10

typedef struct{
   char adress[30];
   int connectionType;
}PageData;
```

Ao final do projeto, seu programa deve rodar as seguintes funções. Note que algumas delas retornam um código de erro. O output pode ser visto na imagem abaixo:

```
Stack* inicializarPilha();
int push(Stack* pilha, char* adress, int connectionType);
int pop(Stack* pilha);

int main(){
   int errorCode;
   Stack* pilha;
   pilha = inicializarPilha(); //Pilha criada
   errorCode = push(pilha, "http://icen.ufpa.br", 2);
   errorCode = push(pilha, "http://www.youtube.com", 2);
   errorCode = push(pilha, "https://www.facebook.com", 1);
   errorCode = pop(pilha);
   errorCode = push(pilha, "https://drive.google.com", 1);
   errorCode = pop(pilha);
   errorCode = pop(pilha);
   errorCode = pop(pilha);
}
```

a) Crie uma struct para abrigar a pilha e defina-a como tipo Stack. Essa struct deve conter o array correspondente à LIFO e um int para guardar o topo da pilha.

b) Escreva a função inicializarPilha que aloca memória para a pilha.

c) Escreva a função push para adicionar um elemento à pilha. Se necessário, utilize a função strcpy(char* destino, const char* origem), da biblioteca padrão, para copiar strings.

d) Escreva a função pop para remover e mostrar o último endereço da pilha.

2) Sejam os inteiros 1, 2, 3 e 4 que são lidos nesta ordem para serem colocados numa pilha. Considerando-se todas as possíveis sequencias de operações Empilha e Desempilha decida quais das 24 (4!) permutacões possíveis podem ser obtidas como saıda da pilha. Por exemplo, a permutacão 2, 3, 1, 4 pode ser obtida da seguinte forma: Empilha 1, Empilha 2, Desempilha 2, Empilha 3, Desempilha 3, Desempilha 1, Empilha 4, Desempilha 4.

Ideia fundamental

Uma pilha segue a regra **LIFO** (*Last In, First Out*): o último elemento empilhado é o primeiro a sair.

Os elementos chegam obrigatoriamente nesta ordem:

$$1 \rightarrow 2 \rightarrow 3 \rightarrow 4$$

Podemos escolher, a qualquer momento:

- **Empilhar (PUSH)** o próximo elemento;
- **Desempilhar (POP)** o elemento que está no topo.

Exemplo

Para obter:

$$2,3,1,4$$

fazemos:

1. PUSH(1)
2. PUSH(2)
3. POP → **2**
4. PUSH(3)
5. POP → **3**
6. POP → **1**
7. PUSH(4)
8. POP → **4**

Portanto, é possível.

3)Considere uma Árvore Binária de Busca (ABB) cujos nós armazenam valores inteiros distintos. Implemente, em linguagem C, um programa que mantenha uma ABB e disponibilize as operações descritas a seguir.

1. Inserção

Implemente abb_insere() para inserir um novo elemento. Valores menores que a chave do nó devem ser armazenados na subárvore esquerda e valores maiores, na direita.

2. Busca

Implemente abb_busca() de forma recursiva, retornando o endereço do nó encontrado ou NULL.

3. Remoção

Implemente abb_remove() considerando obrigatoriamente os três casos: remoção de nó folha; remoção de nó com apenas um filho; remoção de nó com dois filhos. No caso de dois filhos, utilize como substituto o predecessor, isto é, o maior elemento da subárvore esquerda.

4. Altura

Implemente abb_altura() para determinar a altura da árvore. Considere uma árvore contendo apenas a raiz com altura 0.

5. Quantidade de folhas

Implemente abb_folhas() para determinar recursivamente o número de nós folha.

6. K-ésimo menor elemento

Implemente a função abaixo. Ela deve armazenar em valor o k-ésimo menor elemento da ABB. Não é permitido transformar a árvore em um vetor ou utilizar qualquer estrutura auxiliar para armazenar os elementos.

int abb_kesimo_menor(Arv *raiz, int k, int *valor);

7. Verificação de ABB

Implemente eh_abb() que retorne 1 se a árvore satisfaz as propriedades de uma ABB e 0 caso contrário. Não compare apenas cada nó com seus filhos imediatos; a função deve verificar corretamente toda a árvore.

8. Percurso

Implemente o percurso em ordem, imprimindo os elementos em ordem crescente.

Estrutura sugerida

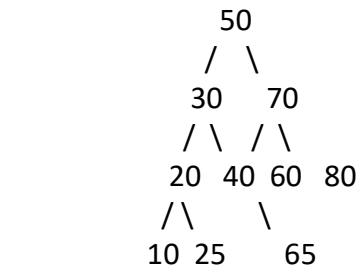
Utilize a seguinte estrutura:

```
typedef struct arv {
   int info;
```

```
   struct arv *esq;
   struct arv *dir;
} Arv;
```

Árvore inicial

O programa principal deve construir a seguinte árvore por meio de inserções sucessivas:



Sequência de operações

1. Inserir 35
2. Inserir 75
3. Remover 20
4. Remover 70
5. Remover 50
6. Inserir 55
7. Inserir 15

Tarefas adicionais

a) Desenhe a árvore após cada operação de remoção.
b) Informe o percurso em ordem da árvore ao final das operações.
c) Determine a altura da árvore final.
d) Determine o número de folhas.
e) Determine o 5º menor elemento utilizando abb_kesimo_menor().
f) Explique detalhadamente como sua implementação de abb_remove() trata a remoção do nó 70 e posteriormente do nó 50, destacando o uso do predecessor.

4) Considere uma Árvore AVL, isto é, uma árvore binária de busca que deve permanecer balanceada após cada operação de inserção e remoção.

Dados de entrada
Insira, nesta ordem, os seguintes valores:
50, 30, 70, 20, 40, 10, 25, 35, 45, 60, 80, 90, 85

Tarefas
a) Defina, em C, a estrutura do nó da árvore AVL contendo, no mínimo: chave; ponteiro para o filho esquerdo; ponteiro para o filho direito; e altura do nó.

b) Implemente as funções:

altura(No *no);

fatorBalanceamento(No *no);

rotacaoDireita(No *y);

rotacaoEsquerda(No *x); e inserir(No *raiz, int chave).

c) Durante a inserção dos elementos, identifique todas as situações em que ocorre desbalanceamento. Para cada situação, informe: o nó que ficou desbalanceado; seu fator de balanceamento; o tipo de caso ocorrido (LL, RR, LR ou RL); e a rotação necessária.

d) Desenhe a árvore após a inserção de cada elemento que provocar uma rotação.

e) Ao final da construção da árvore, apresente: percurso pré-ordem; percurso em-ordem; percurso pós-ordem; altura da árvore; e fator de balanceamento de cada nó.

f) Implemente também uma função para remover um elemento da AVL: No* remover(No *raiz, int chave). Remova, nesta ordem: 30, 70 e 50. Após cada remoção, mostre a árvore resultante, calcule os fatores de balanceamento, identifique se foi necessária alguma rotação e indique o tipo de rotação realizada.

Desafio adicional
Explique por que uma Árvore AVL pode apresentar desempenho O(log n) para busca, inserção e remoção, enquanto uma Árvore Binária de Busca convencional pode chegar a O(n).
Observação importante
Não é permitido simplesmente reconstruir a árvore a partir de uma sequência ordenada. O balanceamento deve ser realizado dinamicamente, utilizando as rotações AVL apropriadas.